

You can also split a string that has a separator using the *s-split* function. For example:

```
(s-split "\\." "192.168.1.100")
("192" "168" "1" "100")
```

The *s-join* function accepts a separator and combines a list of strings provided as input. An example is shown below:

```
(s-join ":" '("28" "24" "ff"))
"28:24:ff"
```

Predicate functions

The *s.el* library provides a number of predicate functions. Two strings can be checked if they are equal using the *s-equals?* function. If there is a match, *t* (true) is returned, otherwise, *nil* is returned. A couple of examples are shown below:

```
(s-equals? "xyz" "XYZ")
nil
```

```
(s-equals? "xyz" "xyz")
t
```

The source code of *s-equals?* from the *s.el* library is as follows:

```
(defun s-equals? (s1 s2)
  "Is S1 equal to S2?"

  "This is a simple wrapper around the built-in string-equal."

  (declare (pure t) (side-effect-free t))
  (string-equal s1 s2))
```

It basically uses the built-in *string-equal* function to compare if two strings are equal or not. You can check if a string is empty using the *s-blank?* function as shown below:

```
(s-blank? "")
t
```

```
(s-blank? nil)
t
```

```
(s-blank? "Beta")
nil
```

A non-empty string will return true *t* when used with the *s-present?* function. If the string is empty, then *nil* is returned.

```
(s-present? "")
nil
```

```
(s-present? nil)
nil
```

```
(s-present? "Beta")
t
```

The *s-ends-with?* function checks if the input string terminates with the suffix, whereas, the *s-starts-with?* function ascertains if the input string begins with the prefix. A couple of examples to demonstrate the function usage are given below:

```
(s-ends-with? ".txt" "README.txt")
t
```

```
(s-starts-with? "READ" "README.txt")
t
```

A *needle* string is given as input to the *s-contains?* function to check if it exists in the input string. For example:

```
(s-contains? "main" "src/main.js")
t
```

You can check if an input string has only lowercase characters using the *s-lowercase?* function, and the *s-uppercase?* function to see if it has only uppercase characters. A few examples are given below:

```
(s-lowercase? "/etc/resolv.conf")
t
```

```
(s-lowercase? "Documents")
nil
```

```
(s-uppercase? "/etc/resolv.conf")
nil
```

```
(s-uppercase? "FOO")
t
```

The *s-mixedcase?* function checks to see if a string has both upper and lower case characters in it. For example:

```
(s-mixedcase? "Ping Pong")
t
```

The *s-capitalized?* function ascertains if the first character of the input string has a capital letter. A couple of examples are given below:

```
(s-capitalized? "foo bar")
nil
```